# TGT: Temporal Gait Transformer for Analyzing Biomechanics of Walking Abnormalities

Your Name
Your Affiliation

## Abstract

We present the Temporal Gait Transformer (TGT), a classification model that takes raw smart insole sensor data and identifies five distinct walking activities: normal walking, injury walking, stepping, swaying, and jumping. The input data comes from pressure sensors, accelerometers, and orientation quaternions on both feet. After merging left and right insole streams into synchronised rows at 60 Hz, we segment the recording into overlapping windows of 20 timesteps (approximately 0.33 seconds each). Each timestep carries 50 sensor features. The TGT projects every timestep into a 128 dimensional embedding, adds a fixed sinusoidal positional encoding, prepends a learnable classification token, and passes the full sequence through four Transformer encoder layers with multi head self attention. A final linear head maps the classification token to five class probabilities. The model is trained with cross entropy loss and the AdamW optimiser over 60 epochs with cosine annealing of the learning rate. We evaluate on a held out test split from the training subjects as well as a fully external validation set collected from a separate group of subjects.

## 1 Data Preprocessing

### 1.1 Data Types

Data is collected from smart insoles on both feet of each subject. At every time step, the insoles measure pressure at multiple locations and provide accelerometer, gyroscope, and magnetometer readings, along with a computed quaternion and an insole label (right or left). Redundant fields include internal time stamps for each subdevice and battery status. A detailed list of all per-timestep features is provided in Table 4 in the Appendix.

### 1.2 Separating Input Data

The raw data for each walking condition is first separated into right and left insole streams using the insole identifier $ele_36$. We then compare the time stamps of rows from the right and left insoles and match them into a single combined row when the difference in recording times satisfies the constraint $\delta t < 0.02\,\mathrm{s}$. Rows that do not satisfy this constraint are discarded. We remove redundant sub-time fields and drop gyroscope and magnetometer readings, because this information is already captured by the quaternions. The resulting dataset, with all retained channels, is summarised in Table 5 in the Appendix. Each remaining row is a full entry for both feet, containing 50 features in total: 18 pressure values, 3 accelerometer axes, and 4 quaternion components per foot.

### 1.3 Sampling Rate

After merging, the data is resampled to a fixed 60 Hz. If the merged stream has a frame rate below 60 Hz we upsample by linear interpolation. If it is above 60 Hz we downsample by selecting evenly spaced indices. This ensures that every recording, regardless of the original device clock, produces a uniform 60 samples per second stream.

### 1.4 Windowing

We define a window parameter $W$ as the number of consecutive rows provided to the model. In our configuration we use $W = 20$, which corresponds to an input segment of approximately 0.33 seconds. The stride between consecutive windows is set to $S = 5$, giving a 75% overlap. This overlap produces more training samples

and ensures that transient gait events are captured even if they fall near a window boundary. Each window is therefore a matrix of shape $20 \times 50$.

### 1.5 Attaching Outcomes

For each input window we attach a walking label from the following set of classes, summarised in Table 1:

### 1.6 Split

The labelled windows are randomly split into three sets: 80% for training, 10% for validation, and 10% for testing. The validation set is used during training to monitor generalisation and to select the best checkpoint. A separate external validation set was collected independently from Group 10 subjects who did not appear in the training data. This external set contains four of the five classes (Normal walking, Injury walking, Stepping, Swaying) and is used as an additional held out evaluation after training is complete.

## 2 Methodology

The architecture of the TGT follows a standard Transformer encoder pipeline adapted for time series classification. The complete flow is: linear projection, positional encoding, classification token, Transformer encoder stack, layer normalisation, and a linear classification head. We describe each stage below.

### 2.1 Linear Projection to 128 Dimensions

The input to the model is a window of shape $W \times 50$, where each of the $W = 20$ rows is a single timestep with 50 sensor features. For every timestep we apply a learned linear projection $\mathbf{z}_t = W_{\text{proj}}\,\mathbf{x}_t + \mathbf{b}_{\text{proj}}$ that maps the 50 dimensional input vector $\mathbf{x}_t$ to a 128 dimensional embedding $\mathbf{z}_t$. This projection is shared across all timesteps and is trained end to end with the rest of the model. After this step we have a $20 \times 128$ matrix of token embeddings.

### 2.2 Positional Encoding

Since the Transformer has no built in notion of ordering, we add a fixed positional encoding to each token embedding so the model can distinguish early timesteps from later ones within the window. For a given position $\text{pos} \in \{0, 1, \ldots, W-1\}$ and a dimension index $i \in \{0, 1, \ldots, 63\}$ we compute:

$$PE_{\text{pos},\,2i} = \sin\left(\frac{\text{pos}}{10000^{\,2i/d_{\text{model}}}}\right) \tag{1}$$

$$PE_{\text{pos},\,2i+1} = \cos\left(\frac{\text{pos}}{10000^{\,2i/d_{\text{model}}}}\right) \tag{2}$$

with $d_{\text{model}} = 128$. The denominator $10000^{2i/d_{\text{model}}}$ means that lower dimensions oscillate rapidly and capture fine grained local timing, while higher dimensions oscillate slowly and encode a broader sense of where the timestep sits within the segment. The result is a $20 \times 128$ matrix of positional vectors which we add element wise to the projection matrix.

### 2.3 Classification Token

Before passing the sequence into the Transformer we prepend a special learnable vector called the [CLS] token. This token has the same dimensionality (128) as the timestep embeddings but is not tied to any specific timestep. It is a free parameter that the model learns during training. Its purpose is to serve as an aggregation point: after the Transformer processes the full sequence, the [CLS] token will have attended to every timestep and should contain a summary representation of the entire window. After prepending, the input to the Transformer is a matrix of shape $21 \times 128$ (20 timestep tokens plus 1 classification token).

### 2.4 Transformer Encoder

The core of the model is a stack of $N = 4$ identical Transformer encoder layers. Each layer consists of two sub-blocks.

#### 2.4.1 Multi Head Self Attention

For each of the 21 tokens, the layer computes three vectors: a Query ($Q$), a Key ($K$), and a Value ($V$), each obtained by a learned linear projection. The attention score between

any two tokens is the dot product of one token's query with another token's key, scaled by $\sqrt{d_{\text{model}}}$. These scores are collected into a $21 \times 21$ matrix and passed through a softmax so that each row sums to one. Multiplying this attention matrix by the Value matrix produces a new $21 \times 128$ matrix where each token is now a weighted combination of all other tokens, according to how relevant they are.

We use $H = 4$ attention heads in parallel. Each head operates on a $128/4 = 32$ dimensional slice of the embedding, computes its own attention independently, and the results are concatenated back to 128 dimensions.

#### 2.4.2 Feed Forward Network

After attention, each token passes independently through a two layer feed forward network (FFN). The FFN first projects from 128 to 512 dimensions (a 4× expansion), applies a Gaussian Error Linear Unit (GELU) activation, and then projects back to 128 dimensions. Residual connections and layer normalisation are applied around both the attention sub-block and the FFN sub-block.

A dropout rate of 0.1 is applied inside both the attention and FFN sub-blocks. The four encoder layers are applied sequentially, and each one refines the token representations further.

### 2.5 Layer Normalisation and Classification Head

After the final encoder layer, we extract the [CLS] token (position 0 of the 21 token sequence). This single 128 dimensional vector is passed through a Layer Normalisation step and then through a fully connected linear layer $\mathbf{o} = W_{\text{cls}}\,\mathbf{h}_{\text{CLS}} + \mathbf{b}_{\text{cls}}$ that outputs 5 values, one for each class. A softmax is applied to these logits to obtain a probability distribution over the five activities.

### 2.6 Training Configuration

The model is trained with standard backpropagation using Cross Entropy Loss between the predicted class probabilities and the one hot encoded ground truth labels. We use the AdamW optimiser, which adds decoupled weight decay to the standard Adam update rule. The specific training parameters are:

The learning rate starts at $3 \times 10^{-4}$ and is decayed following a cosine schedule over the full 60 epochs, reaching near zero by the end of training. We save the model checkpoint that achieves the highest validation accuracy during training and use that checkpoint for all subsequent evaluation.

### 2.7 Model Summary

Table 3 summarises the architectural choices of the TGT.

## 3 Evaluation and Results

*To be completed after training runs.*

## Appendix

### Insole Feature Definitions

### Retained Input Features

| Class ID | Label | Description |
|---|---|---|
| 0 | Normal_walking | Baseline gait pattern without perturbations or injury. Used as reference for distinguishing altered gait behaviours. |
| 1 | Injury_walking | Walking while simulating or experiencing injury (e.g., limping, reduced load on one side), leading to asymmetric pressure and motion patterns. |
| 2 | Stepping | Repeated stepping in place or step-like movements with frequent foot lifts and contacts but limited forward progression. |
| 3 | Swaying | Predominantly static stance with body sway, where the centre of pressure moves but feet largely remain planted. |
| 4 | Jumping | Dynamic movements with clear take-off and landing phases, characterised by brief loss of contact and high impact forces on landing. |

Table 1: Activity classes used for supervised learning. Each 0.33 s window is assigned one of five class IDs according to the folder name of the source recording, following the mapping implemented in the code.

| Parameter | Value |
|---|---|
| Optimiser | AdamW |
| Learning rate | $3 \times 10^{-4}$ |
| Weight decay | $1 \times 10^{-4}$ |
| LR schedule | Cosine annealing |
| Batch size | 64 |
| Epochs | 60 |
| Dropout | 0.1 |

Table 2: Training hyperparameters used for TGT.

| Component | Value |
|---|---|
| Input features per timestep | 50 |
| Window size ($W$) | 20 |
| Stride ($S$) | 5 |
| Embedding dimension ($d_{\text{model}}$) | 128 |
| Attention heads ($H$) | 4 |
| Encoder layers ($N$) | 4 |
| FFN hidden dimension | 512 |
| Positional encoding | Fixed sinusoidal |
| Classification token | Learnable [CLS] |
| Output classes | 5 |
| Total trainable parameters | $\sim$600k |

Table 3: TGT model architecture summary.

| Signal | Range | Components | Elements | Notes |
|---|---|---|---|---|
| Cap sensing (pressure) | $0 \ldots 800$ | 18 electrodes | Ele_0–Ele_17 | Per-electrode plantar pressure. |
| Accelerometer (x, y, z) | $-1 \ldots 1$ | $x, y, z$ | Ele_18–Ele_20 | 3D linear acceleration. |
| Accelerometer (time) | $> 0$ | timestamp | Ele_21 | **Redundant:** inner accelerometer timestamp (same timestep as row, dropped in preprocessing). |
| Gyroscope (roll, pitch, yaw) | $-600 \ldots 600$ | $r, p, y$ | Ele_22–Ele_24 | Angular velocity in °/s. |
| Gyroscope (time) | $> 0$ | timestamp | Ele_25 | **Redundant:** inner gyroscope timestamp (same timestep as row, dropped in preprocessing). |
| Magnetometer (x, y, z) | $-3000 \ldots 3000$ | $x, y, z$ | Ele_26–Ele_28 | Local magnetic field. |
| Magnetometer (time) | $> 0$ | timestamp | Ele_29 | **Redundant:** inner magnetometer timestamp (same timestep as row, dropped in preprocessing). |
| Quaternion | $-1 \ldots 1$ | $x, y, z, v$ | Ele_30–Ele_33 | Orientation computed from all previous sensors. |
| Quaternion (time) | $> 0$ | timestamp | Ele_34 | **Redundant:** inner quaternion timestamp (same timestep as row, dropped in preprocessing). |
| Unused | — | — | Ele_35 | Not used (reserved). |
| Insole side label | $\{0, 1\}$ | right / left | Ele_36 | Right = 0, Left = 1. |
| Battery status | $0 \ldots 100$ | percentage | Ele_37 | **Redundant:** remaining battery level (not used in the model). |

Table 4: Overview of per-timestep insole features, their numeric ranges, and element indices in the raw data vector.

| Signal | Range | Components | Elements | Rationale for inclusion |
|---|---|---|---|---|
| Cap sensing (pressure) | $0 \ldots 800$ | 18 electrodes | Ele_0–Ele_17 | High-resolution plantar pressure map. Captures stance phase, load distribution, and contact pattern differences across activities (e.g., forefoot vs. heel loading in jumping vs. walking). |
| Accelerometer (linear) | $-1 \ldots 1$ | $x, y, z$ | Ele_18–Ele_20 | Foot-frame linear acceleration. Encodes impact magnitude, swing dynamics, and vertical motion, which are highly discriminative for gait phase transitions and dynamic activities such as jumping. |
| Quaternion (orientation) | $-1 \ldots 1$ | $x, y, z, v$ | Ele_30–Ele_33 | 3D orientation of the insole, fused from inertial and magnetic sensors. Provides a numerically stable, gimbal-lock-free representation of foot pose over time, enabling the model to learn differences in foot rotation patterns between gait types. |

Table 5: Subset of insole signals retained for model input. We keep per-timestep plantar pressure, linear acceleration, and fused orientation (quaternions) for each foot, as these jointly capture contact mechanics, dynamic motion, and foot pose. Redundant timestamps, raw gyroscope/magnetometer channels, and battery state are excluded from the learning pipeline.